


```

In [1]: ▶ def viterbi(obs, states, start_prob, trans_prob, emit_prob):
    # Viterbi path (psi) stores the backpointers to reconstruct the path
    # Viterbi probability (delta) stores the max probability at each step
    T = len(obs)
    N = len(states)

    # Initialize tables
    # We use dictionaries to map state names (H, C) to values
    delta = [{}]
    psi = [{}]

    # 1. INITIALIZATION (t=0)
    # Probability = Initial Prob * Emission Prob
    for s in states:
        delta[0][s] = start_prob[s] * emit_prob[s][obs[0]]
        psi[0][s] = None

    # Print initial step for verification
    print(f"--- Initialization (Obs: {obs[0]}) ---")
    for s in states:
        print(f"Delta[{s}] = {delta[0][s]:.4f}")

    # 2. RECURSION (t=1 to T-1)
    for t in range(1, T):
        delta.append({})
        psi.append({})
        print(f"\n--- Step {t+1} (Obs: {obs[t]}) ---")

        for s_next in states:
            # Calculate probability for every possible previous state
            # prob = (delta_prev * trans_prob) * emit_prob
            max_prob = 0
            max_state = None

            for s_prev in states:
                # This calculates the probability of the path ending at
                # multiplied by the transition to s_next
                p = delta[t-1][s_prev] * trans_prob[s_prev][s_next]

                if p > max_prob:
                    max_prob = p
                    max_state = s_prev

            # Apply emission probability for the current observation
            delta[t][s_next] = max_prob * emit_prob[s_next][obs[t]]
            psi[t][s_next] = max_state

        print(f"  Path to {s_next}: came from {max_state} (prob before: {max_prob:.4f})")

    # 3. TERMINATION
    # Find the state with the maximum probability at the final time step
    best_last_state = max(delta[T-1], key=delta[T-1].get)
    max_probability = delta[T-1][best_last_state]

    print(f"\n--- Termination ---")
    print(f"Max probability: {max_probability}")
    print(f"Final state: {best_last_state}")

    # 4. PATH BACKTRACKING
    # Start with the best last state and follow the pointers back
    best_path = [best_last_state]

```

```

        current_state = best_last_state

    for t in range(T-1, 0, -1):
        current_state = psi[t][current_state]
        best_path.insert(0, current_state)

    return best_path, max_probability

# --- DEFINING THE MODEL PARAMETERS FROM THE IMAGE ---

# States
states = ['H', 'C']

# Observations (The sequence we want to decode)
observations = [3, 1, 3]

# Initial Probabilities (Pi)
start_probability = {
    'H': 0.8,
    'C': 0.2
}

# Transition Probabilities (A)
# P(Next | Current)
transition_probability = {
    'H': {'H': 0.7, 'C': 0.3},
    'C': {'H': 0.4, 'C': 0.6}
}

# Emission Probabilities (B)
# P(Observation | State)
emission_probability = {
    'H': {1: 0.2, 2: 0.4, 3: 0.4},
    'C': {1: 0.5, 2: 0.4, 3: 0.1}
}

# --- RUN THE ALGORITHM ---
print("Running Viterbi Algorithm...")
print("-" * 30)
path, prob = viterbi(observations, states, start_probability, transition_probability)

print("-" * 30)
print(f"Most Likely State Sequence: {path}")

```

```
Running Viterbi Algorithm...
-----
--- Initialization (Obs: 3) ---
Delta[H] = 0.3200
Delta[C] = 0.0200

--- Step 2 (Obs: 1) ---
  Path to H: came from H (prob before emission: 0.22400) -> Final Delta: 0.04480
  Path to C: came from H (prob before emission: 0.09600) -> Final Delta: 0.04800

--- Step 3 (Obs: 3) ---
  Path to H: came from H (prob before emission: 0.03136) -> Final Delta: 0.01254
  Path to C: came from C (prob before emission: 0.02880) -> Final Delta: 0.00288

--- Termination ---
Max probability: 0.012544000000000003
Final state: H
-----
Most Likely State Sequence: ['H', 'H', 'H']
```

In []: ▶